THE NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, LAHORE
CS-4031 Compiler Construction — Fall 2026

Lecture 5

Finite Automata: Deterministic and Nondeterministic; Thompson's Construction (Regular Expression → NFA)

CLO(s) covered:	CLO 1, CLO 2
Dragon Book [DB]:	§3.6 (Recognition of Tokens), §3.7 (Finite Automata; From a Regular Expression to an Automaton), Alg. 3.23 (Thompson's Construction)
Other references:	[AP] Modern Compiler Implementation, §2.3–2.4; [HMU] Introduction to Automata Theory, Languages, and Computation, Ch. 2

Lecture Roadmap

1. Why a regular expression alone isn't something a computer can *run* – and what we need instead.
2. **Finite automata**, formally: the 5-tuple definition, transition diagrams, and what it means for a string to be "accepted."
3. **DFA vs. NFA**: exactly one choice per step vs. several (or none) – and why Lecture **??**'s Chomsky-hierarchy table already told you these have *equal* power.
4. **Thompson's construction**: a purely mechanical algorithm that builds an NFA directly from the structure of a regular expression, one small piece per operator – worked all the way through on the running example from Lecture **??**, $(a \mid b)^*abb$.

This lecture builds the second arrow in Lecture **??**'s big-picture diagram: regular expression → NFA. The next lecture builds the rest: NFA → DFA → minimized DFA → an actual running lexer.

5.1 Why We Need an Actual Machine

[DB] A regular expression is a piece of notation – it describes a language on paper, but a computer cannot "run" a piece of algebra directly. To actually scan a source file character by character and decide, in real time, which token pattern (if any) the input matches, we need something a processor can execute: a small, fixed set of states, and a rule for how to move between them as each character arrives. That is exactly what a **finite automaton** is, and Lecture **??**'s big-picture diagram (§**??**) already named the first mechanical step that gets us there: **Thompson's construction**, which converts any regular expression into an equivalent finite automaton. This lecture builds that step in full; the next lecture finishes the journey to a working, table-driven lexer.

5.2 Finite Automata, Formally

[DB] Before comparing deterministic and nondeterministic versions, it helps to pin down what a finite automaton actually *is* – the same formal-definition instinct §**??** used for regular languages themselves.

Definition: Finite Automaton

A **finite automaton** is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where:

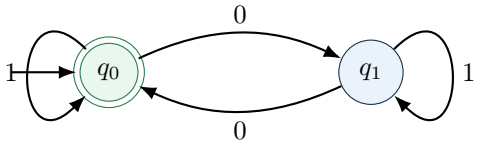
- Q is a finite set of **states**;

- Σ is the input **alphabet** (the same notion from §??);
- δ is the **transition function**, describing how to move between states as input symbols arrive (its exact type differs for DFAs and NFAs – see below);
- $q_0 \in Q$ is the **start state**;
- $F \subseteq Q$ is the set of **accepting** (or *final*) states.

A **transition diagram** is simply this 5-tuple drawn as a picture: a circle per state, a double circle for each accepting state, an arrow (labeled with an input symbol) per transition, and an unlabeled incoming arrow marking the start state – exactly matching the "transition diagram vs. transition table" distinction from §??'s exam-focus box: same underlying automaton, two different ways of writing it down.

Example: Worked Example: Even Number of 0's

Recall from §?? that $1^*(01^*01^*)^*$ describes binary strings with an even number of `0`'s. Here is a two-state automaton for exactly that language: $Q = \{q_0, q_1\}$, $\Sigma = \{0, 1\}$, start state q_0, accepting states $F = \{q_0\}$ (start *and* accept – zero `0`'s is even).



Reading a `0` always *flips* which state you're in (tracking odd/even `0`'s seen so far); reading a `1` never changes the count, so it's a self-loop either way. Try tracing `1010`: start at q_0, read `1` (stay q_0), read `0` (move q_1), read `1` (stay q_1), read `0` (move q_0) – ends at $q_0 \in F$, accepted, matching that `1010` does indeed have an even number (two) of `0`'s.

What "Accepts" Means, Formally

Extend δ to $\hat{\delta}$, a function on *strings* rather than single symbols, by $\hat{\delta}(q, \varepsilon) = q$ and $\hat{\delta}(q, xa) = \delta(\hat{\delta}(q, x), a)$ for a string x followed by one more symbol a – in plain words, "follow the transitions one symbol at a time, starting from q." A string w is **accepted** by the automaton if $\hat{\delta}(q_0, w) \in F$: start at the start state, follow w symbol by symbol, and end up somewhere in the accepting set. The **language of the automaton**, written $L(\text{automaton})$, is the set of all strings it accepts – and for every automaton we build in this course, $L(\text{automaton})$ will turn out to be exactly the regular language we started with.

5.3 DFA vs. NFA: Two Models, Equal Power

[DB] The automaton above had exactly one arrow leaving each state for each symbol – no ambiguity about where to go next. That property has a name.

Definition: Deterministic Finite Automaton (DFA)

A finite automaton is **deterministic** if δ is a genuine function $Q \times \Sigma \to Q$: for every state and every input symbol, there is *exactly one* transition, and no transitions are labeled with ε. At every step, a DFA's next move is completely determined – hence the name.

Real regular expressions, though, are built from *union* – "match r *or* s" – which is naturally ambiguous: from a given state, on a given symbol, there might genuinely be more than one place

a matcher could go. Rather than resolve that ambiguity immediately, it is far more convenient (as Thompson's construction, §5.4, will show) to allow it explicitly in the automaton itself.

Definition: Nondeterministic Finite Automaton (NFA)

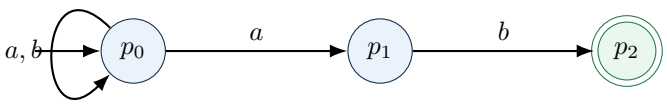
A finite automaton is **nondeterministic** if δ is a function $Q \times (\Sigma \cup \{\varepsilon\}) \to 2^Q$ (the power set of Q): for a given state and input symbol (or ε, the empty string), there can be **zero, one, or many** next states. Two new abilities appear here that a DFA does not have:

- **Multiple choices.** From one state, on one symbol, the automaton may be able to move to several different states at once.
- **ε-transitions.** The automaton may move between states *without reading any input at all* – these are the glue Thompson's construction uses to wire smaller automata together.

A string w is accepted by an NFA if *at least one* sequence of choices, following the symbols of w (and optionally taking any number of ε-transitions in between), ends in an accepting state – not every path has to succeed, just one.

Example: A Small NFA: Strings Ending in `ab`

$Q = \{p_0, p_1, p_2\}$, $\Sigma = \{a, b\}$, start p_0, accept $F = \{p_2\}$:



From p_0, on an `a`, there are *two* valid moves: stay at p_0 (self-loop) or advance to p_1 – exactly the nondeterminism a DFA cannot express directly. To accept `aab`: stay at p_0 reading the first `a` (via the self-loop), then move $p_0 \to p_1$ reading the second `a`, then $p_1 \to p_2$ reading `b` – one successful path is all it takes, even though *guessing wrong* (always taking the self-loop) would never reach p_2.

Exam Focus

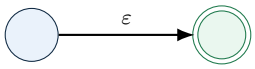
This "one successful path is enough" rule is exactly why simulating an NFA directly is awkward: naively, you'd have to try every possible sequence of choices to know whether *any* of them accepts. Lecture **??**'s subset construction sidesteps this entirely, by tracking *every reachable state at once* as a single set – turning "does some path accept?" into an ordinary DFA question. That NFA-to-DFA conversion is also the constructive proof that NFAs and DFAs recognize exactly the same class of languages (Lecture **??**'s Chomsky-hierarchy table already told you this fact; Lecture **??** shows *why* it's true).

5.4 Thompson's Construction: Regular Expression → NFA

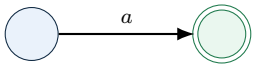
[DB] Thompson's construction (DB Algorithm 3.23) builds an NFA from a regular expression by *structural recursion* – exactly mirroring the six-rule inductive definition of a regular expression itself from §**??**. Every rule below produces a fragment with exactly **one** start state and exactly **one** accept state, which is what makes the rules compose so cleanly: a bigger fragment is always built by wiring together smaller fragments' single start/accept points, never by reaching inside them.

5.4.1 The Five Construction Rules

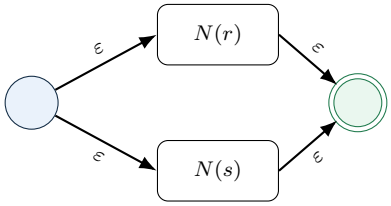
Base case 1: ε. A two-state fragment connected by a single ε-edge.



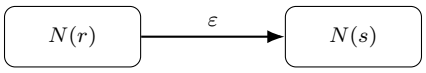
Base case 2: a single symbol $a \in \Sigma$. A two-state fragment connected by one a-edge.



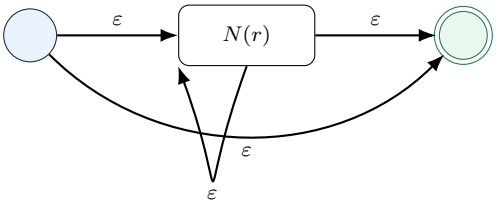
Inductive case: union $r \mid s$. Given fragments $N(r)$ and $N(s)$ (each with its own start/accept), build a new start and new accept state, and wire in *both* old fragments with ε-edges. The old accept states stop being accepting – only the new one is.



Inductive case: concatenation rs. Given $N(r)$ and $N(s)$, connect $N(r)$'s accept state to $N(s)$'s start state with a single ε-edge. $N(r)$'s start becomes the overall start; $N(s)$'s accept becomes the overall accept.



Inductive case: closure r^*. Given $N(r)$, build a new start and new accept state. Wire the new start directly to the new accept (ε, for zero repetitions) and to $N(r)$'s start (ε, to begin a repetition); wire $N(r)$'s accept back to $N(r)$'s start (ε, to loop again) and forward to the new accept (ε, to stop).



Exam Focus

Every fragment above has exactly **one** start and **one** accept state – even after combining fragments. This invariant is what makes the recursion work at all: at every step, you always know exactly which single state to wire the *next* ε-edge into, no matter how deeply nested the regular expression is. Losing track of this (e.g. trying to wire a new edge into "one of the accept states" of a union fragment) is the most common mistake when doing this construction by hand.

5.4.2 Worked Example: Building the NFA for $(a \mid b)^*abb$

[DB] This is exactly the regular expression §?? built up piece by piece with a syntax tree. We now build its NFA the same way, stage by stage, applying one construction rule per stage.

Example: Stage 1 – The Atoms

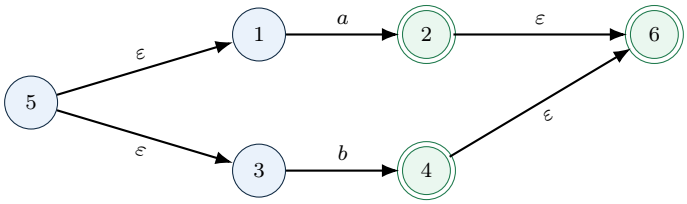
Build the two-state fragments for the individual symbols first. We'll need four of them: one for `a` inside the $(a \mid b)$, one for `b` inside the $(a \mid b)$, and one more each for the trailing `a`, `b`, `b`.



(States 1–2 recognize just `a`; states 3–4 recognize just `b`.)

Example: Stage 2 – Union: $(a \mid b)$

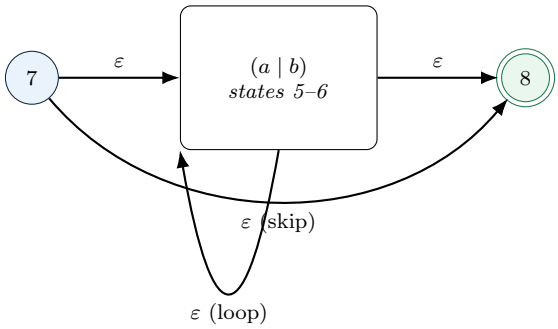
Apply the union rule to the two fragments above: new start state 5, new accept state 6.



(States 2 and 4 are no longer accepting – only the new state 6 is.)

Example: Stage 3 – Closure: $(a \mid b)^*$

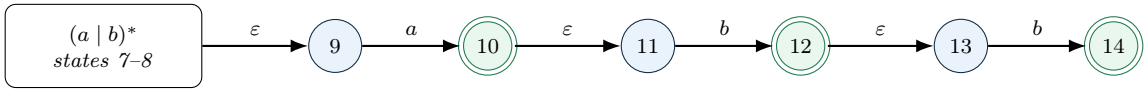
Apply the closure rule to the union fragment above (states 5–6): new start state 7, new accept state 8.



State 7 is now the start of everything built so far; state 8 is the accept state for $(a \mid b)^*$.

Example: Stages 4–6 – Concatenating a, b, b

Three more single-symbol fragments (states 9–10 for `a`, 11–12 for `b`, 13–14 for `b`), concatenated on one at a time: state 8 connects to state 9, state 10 connects to state 11, state 12 connects to state 13 – each via a single ε-edge, exactly the concatenation rule.



The complete NFA for $(a \mid b)^*abb$ has 14 states, start state 7, and a single accept state, 14. Every edge in every stage above is still present – concatenation only ever *adds* an edge, it never removes or renames anything from the fragments it connects.

Example: Tracing `"babb"` Through the Finished NFA

Recall from §**??** that `babb` should be accepted. One successful path: $7 \xrightarrow{\varepsilon} 5 \xrightarrow{\varepsilon} 3 \xrightarrow{b} 4 \xrightarrow{\varepsilon} 6 \xrightarrow{\varepsilon} 8 \xrightarrow{\varepsilon} 9 \xrightarrow{a} 10 \xrightarrow{\varepsilon} 11 \xrightarrow{b} 12 \xrightarrow{\varepsilon} 13 \xrightarrow{b} 14$. Reading off just the symbols consumed along this path – b, a, b, b – reproduces `babb` exactly, and the path ends at state 14, the accept state. One successful path is all an NFA needs.

5.5 Self-Check Questions

Practice – Not Graded

1. Write out the formal 5-tuple $(Q, \Sigma, \delta, q_0, F)$ for the "even number of 0's" automaton in §5.2, giving δ as an explicit table (one row per state, one column per symbol).
2. Using $\hat{\delta}$ from the coreidea box in §5.2, trace whether the string `0110` is accepted by the same automaton. Show each intermediate state.
3. Explain, without using the word "choice," what it means for an NFA's transition function to have type $Q \times (\Sigma \cup \{\varepsilon\}) \to 2^Q$ rather than $Q \times \Sigma \to Q$.
4. In the "strings ending in `ab`" NFA (§5.3), list *all* paths (successful or not) that consume the string `aab`, and confirm exactly one of them ends in the accepting state.
5. Apply Thompson's construction by hand to the regular expression ab^*: draw the fragment for `a`, the fragment for b^*, and the concatenation that joins them.
6. Why does every fragment in Thompson's construction have *exactly one* start state and *exactly one* accept state, even after several rounds of union, concatenation, and closure? What would go wrong with the concatenation rule if a fragment were allowed more than one accept state?
7. For the finished 14-state NFA in §5.4.2, find one string of length 5 that is accepted and one string of length 5 that is *not* accepted, and briefly justify each.

Key Takeaways

- A **finite automaton** is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$; a **transition diagram** is just its picture, and a **transition table** is the same information as an array – interchangeable representations, not different machines.
- A **DFA** has exactly one transition per state per symbol and no ε-edges; an **NFA** allows several transitions (or none) per state per symbol, plus ε-edges that move without consuming input.
- An NFA accepts a string if *at least one* sequence of choices reaches an accepting state – not every path has to succeed.
- **Thompson's construction** builds an NFA from a regular expression using five rules (ε, a single symbol, union, concatenation, closure), each producing a fragment with exactly one start and one accept state – which is exactly what lets the rules compose recursively, mirroring the regular expression's own inductive definition rule for rule.
- The worked $(a \mid b)^*abb$ NFA (14 states) is the same running example from Lecture **??** – Lecture **??** converts this exact automaton into a DFA and then minimizes it, so it is worth understanding this construction solidly before moving on.

References for This Lecture

References

[DB] Aho, A. V., Lam, M. S., Sethi, R., and Ullman, J. D. *Compilers: Principles, Techniques, and Tools*. 2nd ed. Pearson, 2006.

[AP] Appel, A. W. *Modern Compiler Implementation in C/Java/ML*. Cambridge University Press, 2004.

[HMU] Hopcroft, J. E., Motwani, R., and Ullman, J. D. *Introduction to Automata Theory, Languages, and Computation*. 3rd ed. Pearson, 2006.